Flutter Development Essentials

This presentation covers fundamental concepts and essential widgets for building robust Flutter applications.

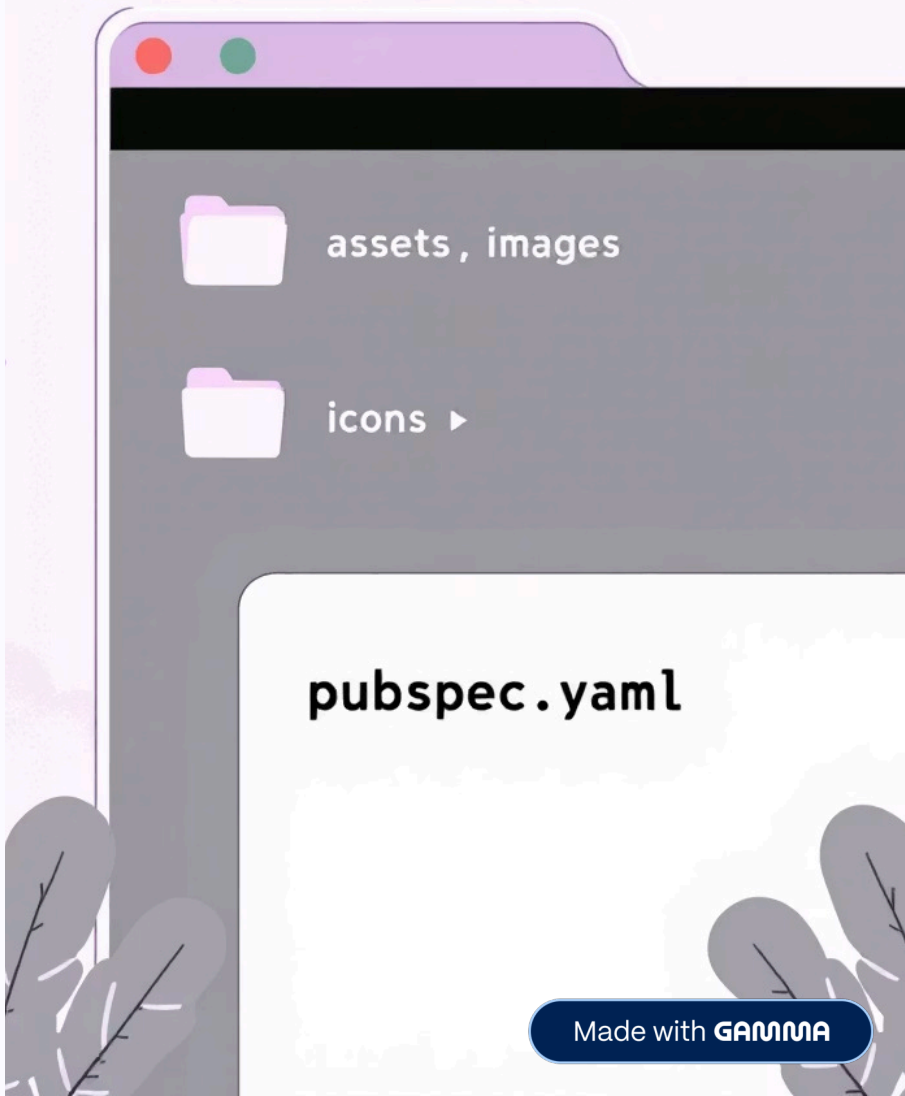
Adding Assets in pubspec.yaml

To include assets like images and icons in your Flutter project, you need to declare them in the `pubspec.yaml` file.

```
flutter:
  assets:
    - assets/images/
    - assets/icons/
```

- Create a folder inside your project (`assets/images`).
- Put your images inside it.
- Declare the path inside `pubspec.yaml`.
- Use it in Flutter with:

```
Image.asset("assets/images/my_photo.png");
```



Widgets: GestureDetector

The `GestureDetector` widget is used to detect various gestures on its child widget.

> Detects gestures like tap, double tap, long press, drag etc.

> Does not show visual feedback.

```
GestureDetector(
 onTap: () {
 print("Tapped!");
 },
 child: Container(
 color: Colors.blue,
 padding: EdgeInsets.all(20),
 child: Text("Tap me"),
 ),
);
```

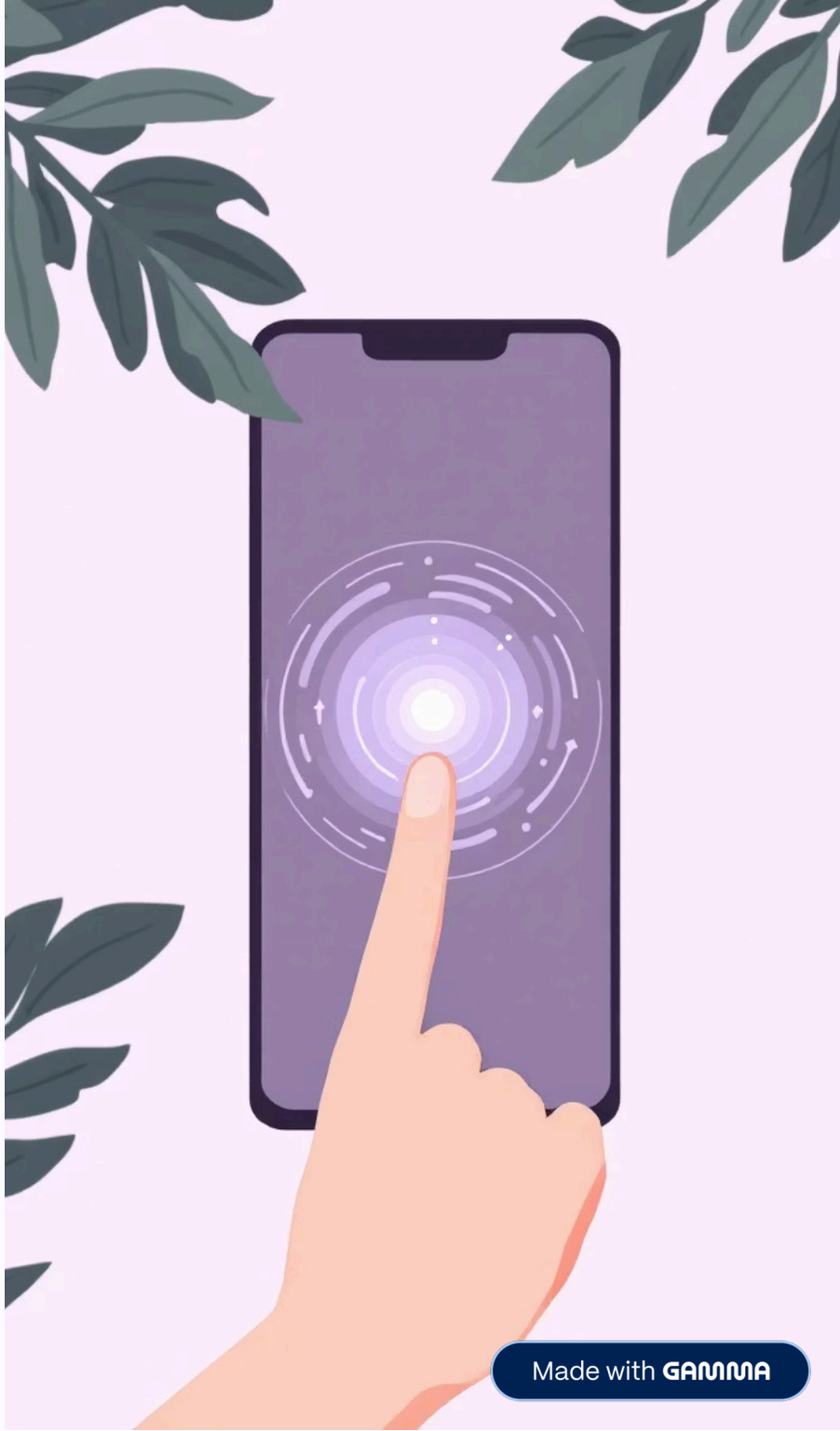
Widgets: InkWell

The `InkWell` widget is similar to `GestureDetector` but provides visual feedback, specifically a ripple effect, adhering to Material Design principles.

Similar to GestureDetector

Shows ripple effect (Material Design feedback).

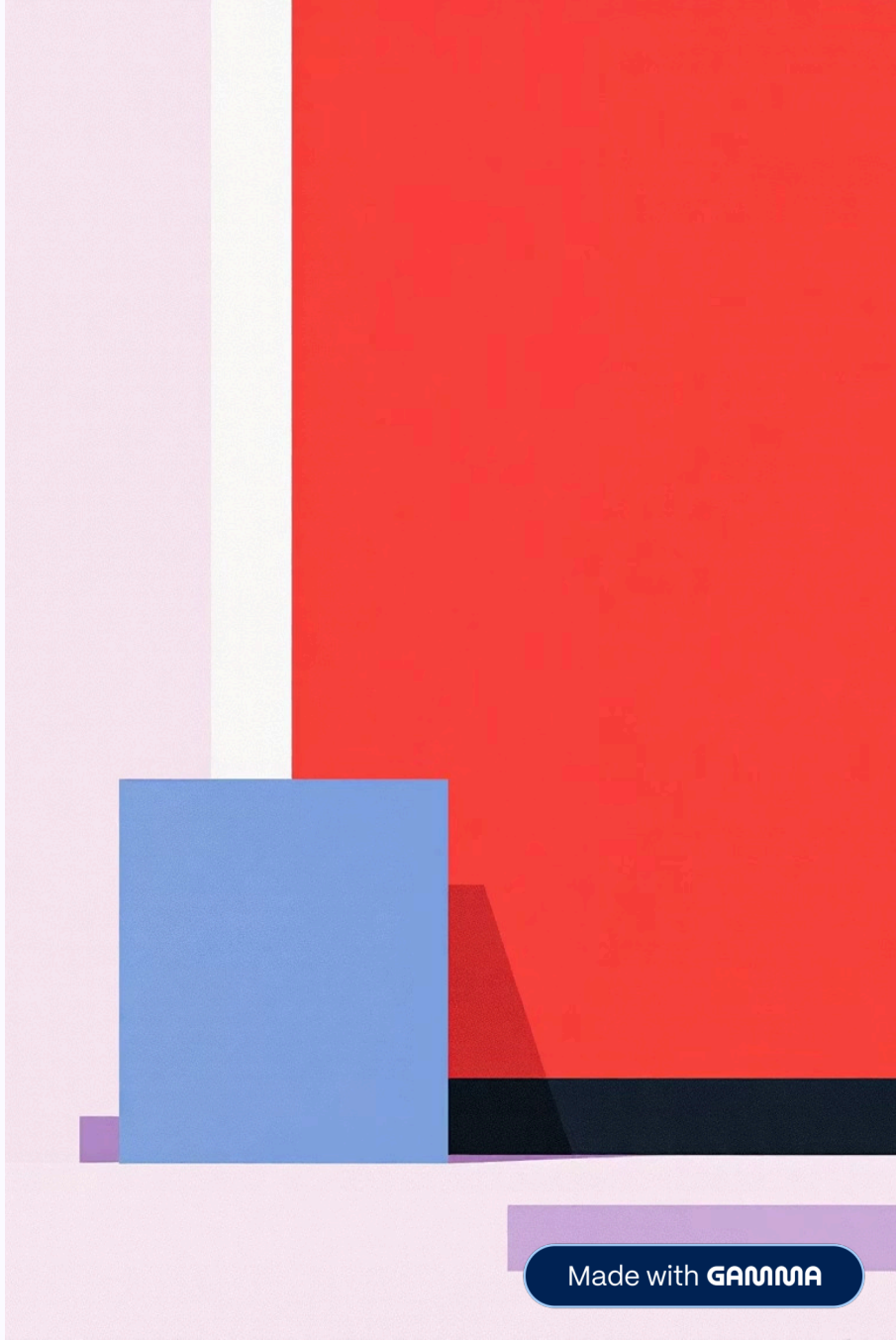
```
InkWell(
 onTap: () {
 print("InkWell tapped!");
 },
 child: Container(
 padding: EdgeInsets.all(20),
 child: Text("Click me"),
 ),
);
```



Widgets: Expanded

The `Expanded` widget is crucial for flexible layouts within `Row` or `Column` widgets, allowing its child to fill available space.

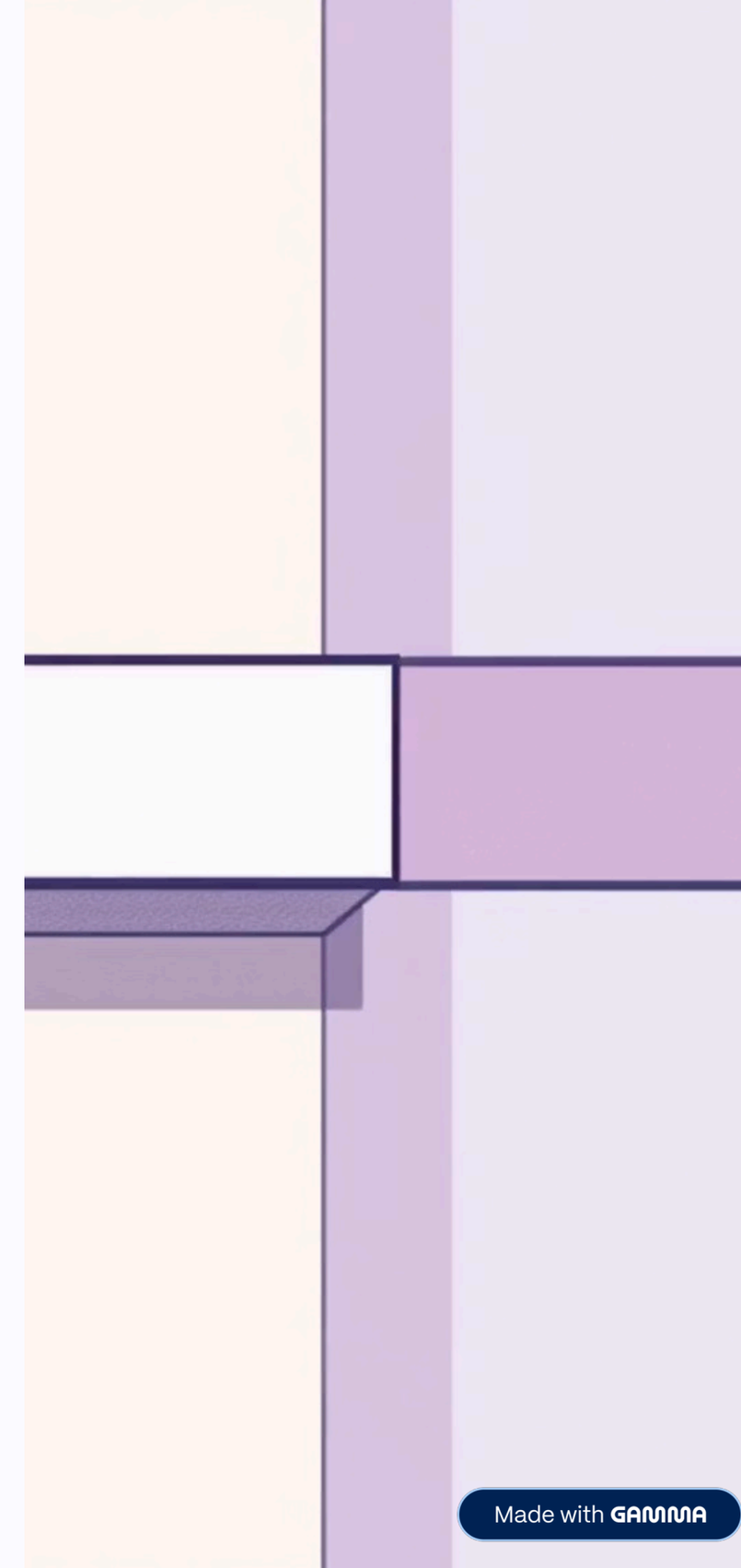
```
Row(
 children: [
 Expanded(
 child: Container(color: Colors.red, height: 50),
 ),
 Container(width: 50, height: 50, color: Colors.blue),
 ],
);
```



Expanded Example 2 (50% – 50%)

When using `Expanded` with equal `flex` values, the space will be divided equally between widgets. This works on all devices and screen sizes.

```
Row(
 children: [
 Expanded(
 flex: 1,
 child: Container(
 height: 100,
 color: Colors.green,
 child: Center(child: Text("Left Half")),
 ),
 ),
 Expanded(
 flex: 1,
 child: Container(
 height: 100,
 color: Colors.orange,
 child: Center(child: Text("Right Half")),
 ),
 ),
 ],
);
```



Widgets: TextFormField

The `TextFormField` widget provides an input field with built-in validation capabilities, essential for user input forms.

```
TextFormField(
 decoration: InputDecoration(
 labelText: "Enter your name",
 border: OutlineInputBorder(),
 prefixIcon: Icon(Icons.person),
 ),
 validator: (value) {
 if (value == null || value.isEmpty) {
 return "Please enter a name";
 }
 return null;
 },
);
```

Image Widget: From Assets

The `Image` widget is used to display images from various sources. One common source is from your project's assets.

```
Image.asset("assets/images/pic.png");
```

Image Widget: From Network

You can also display images directly from a network URL using the `Image.network` constructor.

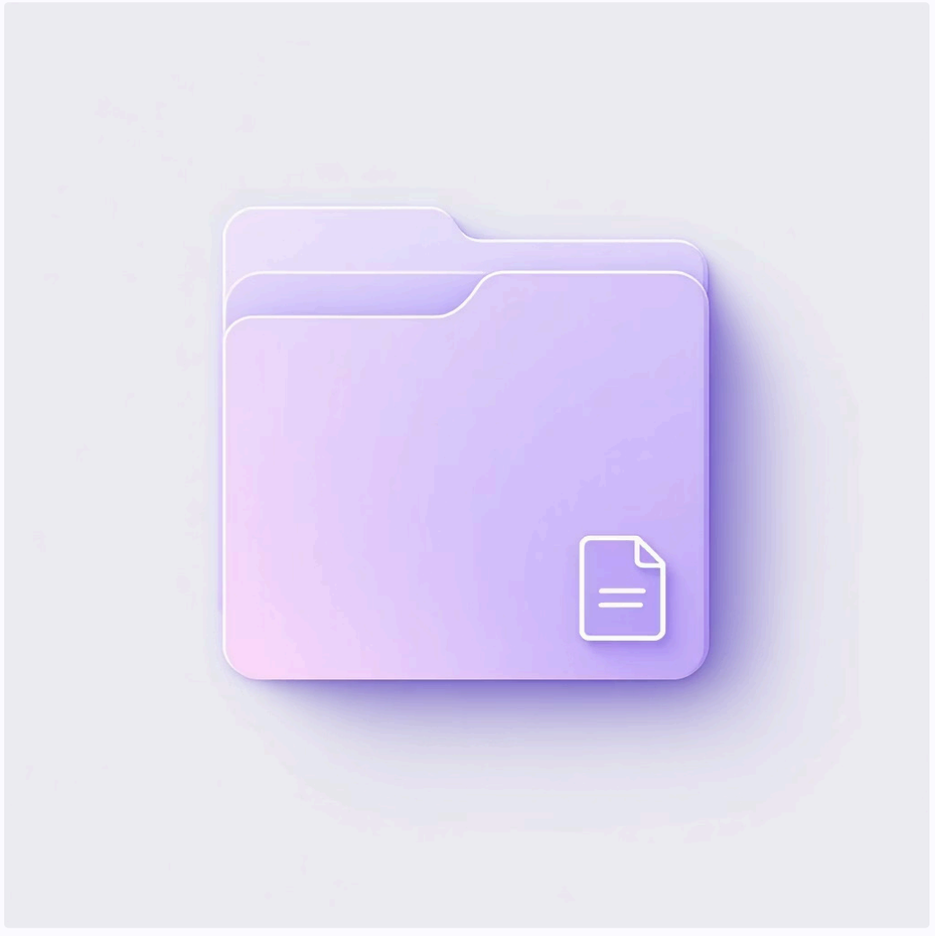
```
Image.network("https://example.com/photo.jpg");
```

Image Widget: From File and Memory

Flutter's `Image` widget offers flexibility to display images from local files or directly from memory (e.g., byte data).

From file

```
Image.file(File("path/to/image.png"));
```



From memory

```
Image.memory(bytes);
```